

视频: <https://juejin.im/post/6844903701459501070>

网站: <http://jspang.com>

一、安装

安装工具

```
1 yum -y install gcc gcc-c++ autoconf pcre-devel make automake
2 yum -y install wget httpd-tools vim
```

建立统计配置目录便于管理 (可选)

1. 进入系统后, 在目录下建立了一个jspang的文件夹。
2. 进入 jspang文件夹 ,命令是 cd jspang。
3. 分别使用mkdir建立 app,backup,download,logs,work文件夹。

我们可以先来查看一下yum是否已经存在, 但是yum源中所保存的版本可能比较低, 所以要更新yum源

```
1 yum list | grep nginx
```

更新yum源 (centos7)

```
1 vim /etc/yum.repos.d/nginx.repo
2
3 // 在文件中增加
4
5 [nginx]
6 name=nginx repo
7 baseurl=http://nginx.org/packages/OS/OSRELEASE/$basearch/
8 gpgcheck=0
9 enabled=1
```

```
1 yum install nginx  安装服务
2 rpm -ql nginx     查看Nginx的安装目录
3 systemctl start nginx.service // 启动
4 nginx -s stop     立即停止
5 nginx -s quit     从容停止服务
6 killall nginx     直接杀死
7 systemctl stop nginx.service  使用系统命令停止
8 systemctl restart nginx.service 重启
9 nginx -s reload   重新载入配置文件
```

二，配置文件解读（vim nginx.conf）

```
1 #运行用户，默认即是nginx，可以不进行设置
2 user nginx;
3 #Nginx进程，一般设置为和CPU核数一样
4 worker_processes 1;
5 #错误日志存放目录
6 error_log /var/log/nginx/error.log warn;
7 #进程pid存放位置
8 pid /var/run/nginx.pid;
9
10 events {
11     worker_connections 1024; # 单个后台进程的最大并发数
12 }
13
14 http {
15     include /etc/nginx/mime.types; #文件扩展名与类型映射表
16     default_type application/octet-stream; #默认文件类型
17     #设置日志模式
18     log_format main '$remote_addr - $remote_user [$time_local] "$request" '
19     '$status $body_bytes_sent "$http_referer" '
20     '"$http_user_agent" "$http_x_forwarded_for"';
21
22     access_log /var/log/nginx/access.log main; #nginx访问日志存放位置
23
24     sendfile on; # 开启高效传输模式
25     #tcp_nopush on; #减少网络报文段的数量
26
27     keepalive_timeout 65; #保持连接的时间，也叫超时时间
28
29     #gzip on; #开启gzip压缩
30
31     include /etc/nginx/conf.d/*.conf; #包含的子配置项位置和文件
```

三，访问权限控制

```
1 location / {
2     allow 45.76.202.231;
3     deny all;
4 }
```

只允许45.76.202.231进行访问，其他的IP是禁止访问的。但是如果我们把deny all指令，移动到allow 45.76.202.231之前，会发生什么那？会发现所有的IP都不允许访问了。这说明了一个问题：就是在同一个块下的两个权限指令，先出现的设置会覆盖后出现的设置（也就是谁先触发，谁起作用）。

四，基于域名设置虚拟主机

五，nginx负载均衡

```
1 # backend是名字，可以任意取
2 upstream backend {
3     ip_hash; # 这里可以不写，不写的话就是轮询算法
4     server x.x.x.x:1023;
5     server x.x.x.x:1024;
6     server x.x.x.x:1025;
7 }
8
9 对特定请求进行分发
10 server {
11     listen 1111;
12     server_name www.siguoya.name;
13
14     location / {
15         proxy_pass http://backend; # 这里要和上面取的名字一样
16     }
17 }
```

这样访问www.siguoya.name这个域名的所有uri都会被负载均衡到backend的这三台服务器上

配置详解

```
1 upstream backend {
2     #down表示当前的服务器暂时不可用，不参与负载均衡
3     server www.aaa.com down;
4     #max_conns=100表示此服务器的最大连接数为100
5     server www.bbb.com:1234 max_conns=100;
6     #weight表示权重，权重越大，被分配到的次数就越多
7     server www.ccc.com weight=3;
8     #backup表示这是预留的备份服务器，只在其他的节点都不可用的时候才启用
9     server www.ddd.com backup;
10    #max_fails在连续超过3次失败后，30秒内不要再分发请求到此服务器，默认fail_timeout为10秒
11    server x.x.x.x:8080 max_fails=3 fail_timeout=30s;
12 }
```

服务器调度详解

- 1, 轮询 按时间顺序逐一分配
- 2, 加权轮询 权重值越大，被分配到的次数就越多
- 3, ip_hash 请求按访问IP的hash结果来分配，同一个IP固定的访问某台后端服务器
- 4, least_conn 哪台服务器连接数少，就分发给哪台服务器
- 5, url_hash 按照访问URL的hash结果来分配，同一URL固定的访问某台后端服务器
- 6, hash_key 自定义的key

经过实际测试，ip_hash并不能完全保持cookies的连续性，最好还是将cookies存放在redis等内存型数据库中

七, location语法

Nginx 将配置按照层级关系,用块状形式进行配置,每当一个请求来临时,nginx服务器就会处理这个请求到底会映射到哪个块配置.在 Nginx 的配置文件中,两种主要的块配置是:一 server块配置, 二 location块配置

server的块配置包含一系列的虚拟 server 配置,多个 server 配置就可以对多个 domain name 的请求,也包括 ip 端口进行处理.

而location配置在server块配置中起着至关重要的作用,决定了URI或者资源请求应该如何被处理,这些URI 请求可以被拆分为多个location的配置

如下就是常用的 location 配置的语法格式,其中modifier是可选的,location_match就是制定 URI 应该去往哪个配置的关键.

```
1 location optional_modifier location_match {  
2     . . .  
3 }
```

Regular expressions(RE)或者字面量都可以用来定义modifier,如果 location 配置中制定了modifier,可能会改变 nginx匹配 location的方式,如下介绍几种最重要的modifier:

(none) 完全没有modifier表示 location会解释为前缀匹配,要确定匹配项,将根据从URI的开头匹配该 location.

= 等号表示当前这个 location 会匹配一个确定的请求,配置什么就匹配什么请求,如果匹配上了,就会停止搜索.

~ 波浪号表示当前这个 location 会当成一个大小写敏感的RE匹配.

~* 波浪号跟星号标识 location 会按照大小写不敏感的 RE 匹配.

^~ 非表达式(RE)匹配,正则表达式将不会生效.

location的匹配顺序

- 1, 首先匹配前缀匹配(没有 RE 表达式),针对当前这个请求,每个前缀匹配都匹配一遍.
- 2, 搜索=匹配,如果当前请求匹配上了,搜索将会停止,直接使用这个这个 location.
- 3, 如果第二步没有匹配上,nginx 会按照如下步骤继续搜索最长前缀匹配:
 - 3.1 如果最长前缀匹配有^~这个modifier,nginx 会停止搜索并直接使用这个 location.
 - 3.2 如果没有使用 ^~,暂存这个 location并且继续搜索.
- 4, 只要最长前缀匹配被暂存和选中,nginx 就会看当前的 location 是否有大小写敏感的 RE(~和~*),第一个匹配上这种会被当做有效的 location来处理这个请求.
- 5, 如果没有 RE 的 location 匹配上,前面暂存的 location 就会被选中来处理这个请求.

这种写法还没有明白

```
1 location ~ ^/api/(v1/ent_configs/.*)$ {  
2     limit_req zone=wx burst=10 nodelay;  
3     proxy_pass http://hikari/$1?$args;  
4 }
```

